

Name: Ameey Mukund Badgajar Class: 50C 08 Batch: A  
Subject: PL - II Roll No.: 07 Exp. No.: 2

Name of the Experiment: \_\_\_\_\_

Performed on: \_\_\_\_\_

Submitted on: \_\_\_\_\_

| Marks | Teacher's Signature<br>with date |
|-------|----------------------------------|
| 10    |                                  |

1. List Data types are:

• Strings (str): immutable and ordered collections of characteristics

• Lists: mutable & ordered collections of elements

• Tuple: immutable and ordered collections of elements

• Range: immutable and ordered sequence of number

2. Indexing is used to access a single element of a sequence its position. Positive indexing starts from - 1

Slicing extracts a part of sequence using the syntax

Incomplete for Theory ☐ Diagrams ☐ Observation Table ☐

Calculations ☐ Graphs ☐ Results ☐ Conclusion ☐

Understanding Through Q/A ☐ Late Submission ☐ Neatness ☐

Teacher's Signature

Difference in Strings & lists:

- in strings indexing / slicing returns characters or substrings

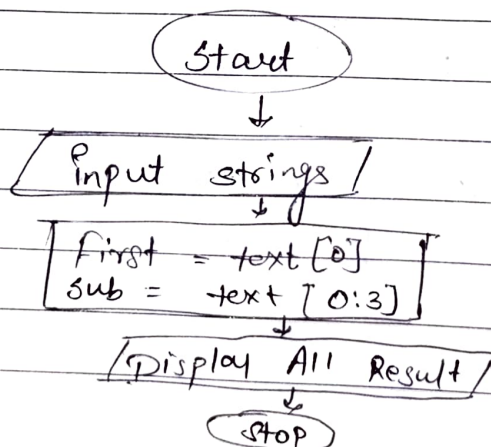
- in lists indexing / slicing returns elements or sublists

Example : strings : "Python" [1:4] → "yth"  
lists : [1, 2, 3, 4] [1:3] → [2, 3]

3. code :-

```
text = "Engineering"
first-char = text[0]
sub-text = text[0:5]
length = len(text)
upper-text = text.upper()
```

```
print ("First character :", first-char)
print ("Slice text :", sub-text)
print ("Length :", length)
print ("Uppercase :", upper-text)
```



4. Difference between equality ( $=$ ) and identity is

- $=$  (Equality operator) checks whether 2 sequences have the same values.

- $is$  (identity operator) checks whether 2 variables refer to the same object in memory.

eg.  $a = [1, 2]$   
 $b = [1, 2] \rightarrow a = b$  is True, but  $a is$   
 $b$  is False

5. • lists allow duplicates and are ordered, but searching is slower

- Sets store only unique values and are fast for membership testing

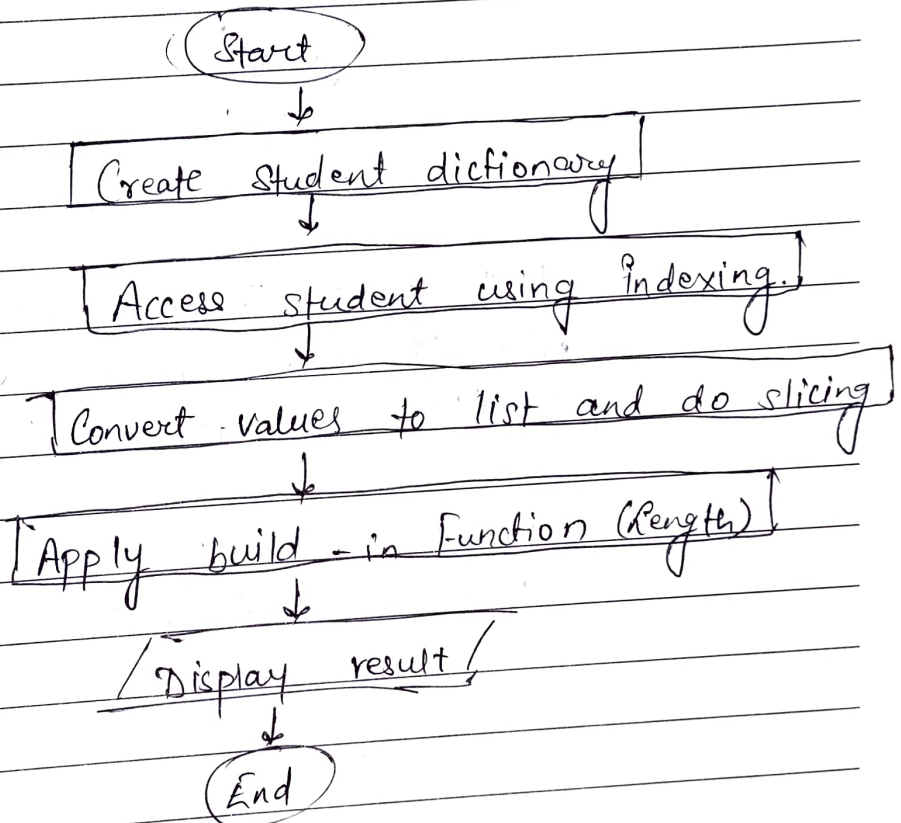
- Dictionaries store data in key-value pairs and allow fast access using keys.

Most efficient: Dictionaries are best for student records because each student can be stored a unique ID as a key, enabling fast lookup, update & retrieve.

6. Program to manage student data.

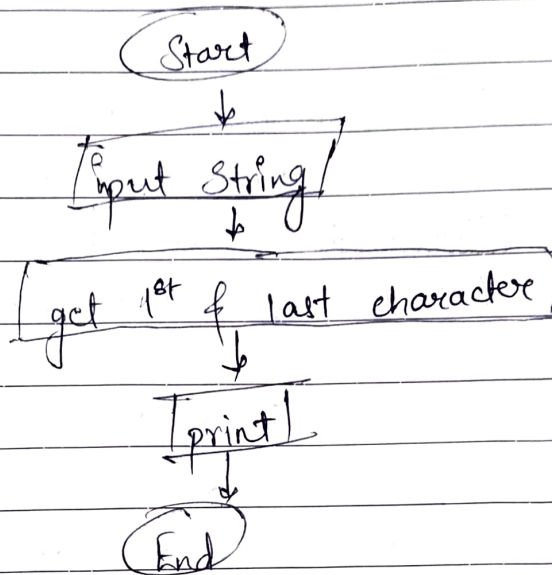
```
student = {  
    101 = "Aman"  
    102 = "Riya"  
    103 = "Karan"  
}
```

```
print ("Student with ID 101 : " student [101]  
name = list (student . values ()  
print ("first two student's , names [0,2]  
print ("Total student : " , len (student))
```





7. Algorithm :
1. Start
  2. input a string
  3. Print 1<sup>st</sup> characteristics using indexing
  4. Print last characteristics using index
  5. End.

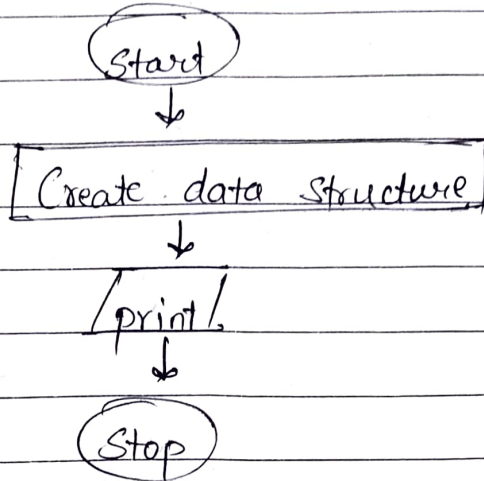


```
code : text = input ("Enter a string:")  
print ("First characteristics : ", text[0])  
print ("last characteristics ", text[-1])
```

output      Enter a string : Python  
              First character : P  
              last character : n

8. Algorithm
1. Start
  2. Create list, tuple, set, dictionary
  3. Display them
  4. Stop.

Flowchart



code :

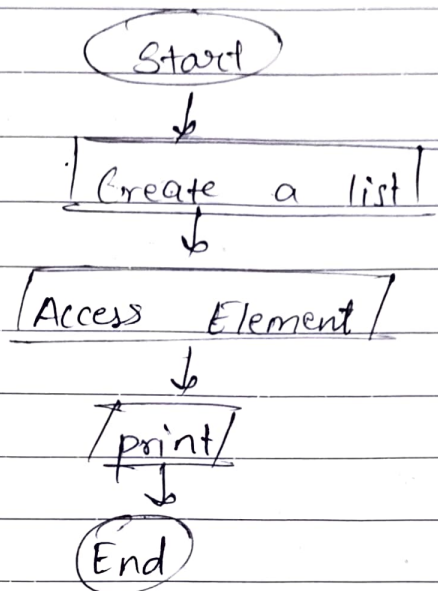
```
lst = [1, 2, 3]
tup = (4, 5, 6)
st = {7, 8, 9}
dct = {"a": 1, "b": 2}
```

output

```
list : [1, 2, 3]
tuple : (4, 5, 6)
set : {7, 8, 9}
Dictionary : {"a": 1, "b": 2}
```

9. Algorithm :
1. Start
  2. Create a list
  3. Access elements using indexing
  4. Print elements
  5. End.

Flowchart :



Code : `Numbers = [10, 20, 30, 40]`

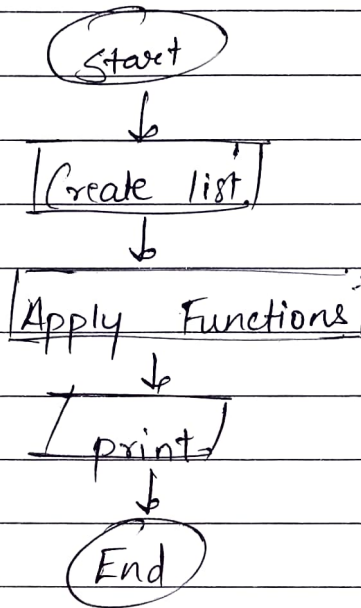
```
print ("First element : " + numbers[0])  
print ("second element : " + numbers[1])  
print ("last element : " + numbers[-1])
```

output

First element : 10  
Second element : 20  
last element : 30

10. Algorithm :
1. Start
  2. Create a list
  3. Apply build-in functions
  4. Print results
  5. End.

Flowchart :



Code :

```
numbers = [5, 10, 15, 20]
print ("Length : ", len (numbers))
print ("Maximum : ", Max (numbers))
print ("the Minimum : ", Min (number))
print ("Sum : ", sum (number))
```

Output :

```
Length : 4
Max : 20
Min : 5
Sum : 50
```